

Light Scattering by Dust Grains with DDSCAT

Sophia Kugler

Summer Student Internship
Christian-Albrechts-Universität zu Kiel
September 2026

Contents

1	Introduction	3
2	Methods and Results	4
2.1	DDSCAT	4
2.2	Solid homogeneous sphere	4
2.2.1	Different materials	5
2.2.2	Different wavelengths	6
2.2.3	Different porosities	6
2.2.4	Oblate shapes	11
2.3	DSHARP Analysis	12
2.3.1	A naive approach	12
2.3.2	Bruggeman rule	14
2.4	Combined Oblateness and Porosity	15
2.5	Different grain sizes	16
2.6	Two materials in one grain	17
3	Conclusion	20
4	Appendix	22
4.1	Python code used in the analysis	22
4.1.1	Reading DDSCAT output and comparison with Mie theory	22
4.1.2	Generation of porous spherical targets	24
4.1.3	Normalization of porous-grain cross sections	27
4.1.4	Generation of compact oblate targets	27
4.1.5	Combined oblateness and porosity	30
4.1.6	DSHARP optical constants	31
4.1.7	Conversion from mass fractions to volume fractions	32
4.1.8	Bruggeman effective-medium calculation	32
4.1.9	Comparison with the optool DSHARP mixture	34
4.1.10	Naive volume-weighted absorption	35
4.1.11	FeS Mie calculation	36
4.1.12	Different grain sizes	36
4.1.13	Explicit Astrosil–water-ice grains	36
4.1.14	Reading DDSCAT material files	39
4.1.15	Two-component Bruggeman calculation	40
4.1.16	Writing the Bruggeman mixture for optool	43
4.1.17	Explicit DDSCAT versus Bruggeman comparison	43
4.1.18	Visualisation of custom targets	44

1 Introduction

The aim of this project is to investigate the light-scattering properties of compact and porous dust grains using the discrete dipole approximation (DDA) implemented in DDSCAT.

Before applying DDSCAT to porous and therefore non-spherical grain structures, the numerical method is first tested for a compact homogeneous sphere. For this geometry, an exact Mie solution is available and can be used as a reference. The required dipole resolution is determined by comparing DDSCAT calculations with Mie calculations performed with `optool`. The dependence on material and wavelength is then investigated before the method is applied to porous grains.

2 Methods and Results

2.1 DDSCAT

DDSCAT represents a target by a three-dimensional array of polarizable point dipoles. Increasing the number of dipoles improves the spatial resolution of the target but also increases the computational cost.

For the compact-sphere calculations, the DDSCAT target `ELLIPSOID` was used with equal dimensions in all three directions. The physical grain radius was kept fixed while the number of dipoles was varied.

The applicability of the DDA can be characterized by the parameter

$$|m|kd, \quad (1)$$

where m is the complex refractive index, $k = 2\pi/\lambda$ is the wavenumber, and d is the dipole spacing. Smaller values correspond to a better spatial resolution of the electromagnetic field.

2.2 Solid homogeneous sphere

For a homogeneous spherical grain, the scattering problem can be solved exactly using Mie theory. These calculations were performed with `optool` (<https://github.com/cdominik/optool>) and used as reference solutions for the DDSCAT calculations.

DDSCAT directly provides the dimensionless efficiency factors

$$Q_{\text{abs}}, \quad Q_{\text{sca}}, \quad Q_{\text{ext}}, \quad (2)$$

as well as the asymmetry parameter g . In contrast, `optool` provides the absorption and scattering mass opacities κ_{abs} and κ_{sca} in $\text{cm}^2 \text{g}^{-1}$, together with g .

For a compact spherical grain, the opacities were converted to efficiency factors using

$$Q = \frac{\kappa m_{\text{grain}}}{\pi a^2} = \frac{4}{3} \rho a \kappa, \quad (3)$$

where a is the grain radius and ρ is the material density. The extinction efficiency is then obtained from

$$Q_{\text{ext}} = Q_{\text{abs}} + Q_{\text{sca}}. \quad (4)$$

After this conversion, the `optool` Mie results can be compared directly with the corresponding DDSCAT quantities. The absolute relative error was calculated as

$$\epsilon_X = \left| \frac{X_{\text{DDSCAT}} - X_{\text{Mie}}}{X_{\text{Mie}}} \right| \times 100\%, \quad (5)$$

where X represents Q_{abs} , Q_{sca} , Q_{ext} , or g .

2.2.1 Different materials

Two dust materials were considered: astronomical silicate (**astrosil**) and the DSHARP dust mixture. The calculations were performed for spheres with radius $a = 1\,\mu\text{m}$ at a wavelength of $\lambda = 1\,\mu\text{m}$. DDSCAT version 7.3.4 was used for the discrete dipole calculations.

For the DDSCAT calculations, the physical grain size was kept fixed while the number of dipoles was increased (see Fig. 1).

DDSCAT represents the grain by N cubic volume elements with dipole spacing d . Each dipole therefore represents a volume d^3 , so that the total material volume is

$$V = Nd^3. \quad (6)$$

DDSCAT defines the effective radius a_{eff} as the radius of a sphere with the same material volume,

$$a_{\text{eff}} = \left(\frac{3Nd^3}{4\pi} \right)^{1/3}. \quad (7)$$

For the compact spherical targets used here, $a_{\text{eff}} = 1\,\mu\text{m}$ was kept fixed. The physical diameter is therefore $D = 2a_{\text{eff}} = 2\,\mu\text{m}$. Increasing the number of dipoles does not change the physical grain size; instead, it decreases the dipole spacing according to

$$d = a_{\text{eff}} \left(\frac{4\pi}{3N} \right)^{1/3}. \quad (8)$$

The geometrical resolution can equivalently be described by D/d , i.e. the number of dipole spacings across the particle diameter. For a spherical grain,

$$\frac{D}{d} = 2 \left(\frac{3N}{4\pi} \right)^{1/3}. \quad (9)$$

Thus, increasing N corresponds to a smaller dipole spacing and a finer representation of the same $1\,\mu\text{m}$ grain. The exact value of N is the number of occupied lattice sites in the discrete DDSCAT target.

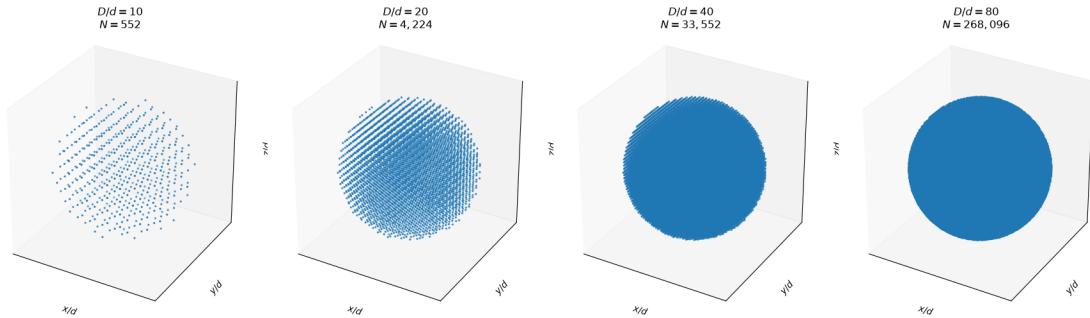


Figure 1: DDSCAT representation of the same spherical grain using different numbers of dipoles.

Figure 2 shows that the convergence is not strictly monotonic at low resolution. Nevertheless, with approximately 8×10^3 dipoles, all quantities agree with the Mie solution to within about 3.5%. At approximately 1.8×10^5 dipoles, the errors in Q_{abs} , Q_{sca} , Q_{ext} , and g are all below 1%. The highest-resolution calculation, using approximately 2.7×10^5 dipoles, gives errors below 0.7% for all four quantities.

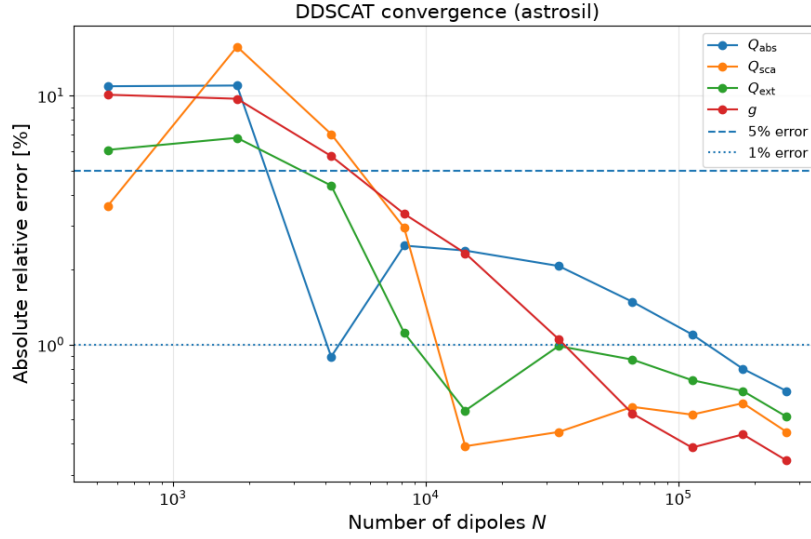


Figure 2: Absolute relative error of the DDSCAT results compared with the Mie reference solution calculated with `optool`. The physical grain radius and wavelength were kept fixed at $a = 1 \mu\text{m}$ and $\lambda = 1 \mu\text{m}$, while the number of dipoles was increased. Astronomical silicate was used as the grain material.

The same convergence test was repeated using the DSHARP dust mixture in order to investigate whether the required dipole resolution depends on the optical properties of the material.

2.2.2 Different wavelengths

After determining a suitable dipole resolution, the wavelength dependence was investigated. The grain radius was kept fixed at $a = 1 \mu\text{m}$ and a resolution of $D/d = 70$ was used, corresponding to $N = 179\,944$ dipoles for the compact sphere. The wavelength was varied from $0.3 \mu\text{m}$ to $100 \mu\text{m}$.

The calculations were first performed for astronomical silicate and then repeated for the DSHARP dust mixture. The resulting Q_{abs} , Q_{sca} , Q_{ext} , and g were investigated as functions of wavelength. Figure 4 shows the wavelength dependence of the optical efficiencies for the two materials. Panel (a) corresponds to the DSHARP dust mixture, while panel (b) shows the results for astronomical silicate.

2.2.3 Different porosities

To investigate the effect of porosity, grains with porosities between 0% and 60% were compared while keeping the spherical outer envelope fixed at $a_{\text{outer}} = 1 \mu\text{m}$. Since DDSCAT defines the effective radius from the volume of solid material,

$$a_{\text{eff}} = \left(\frac{3V_{\text{material}}}{4\pi} \right)^{1/3}, \quad (10)$$

a_{eff} decreases when the porosity increases, even though the outer size of the grain stays the same.

DDSCAT reports the absorption efficiency as

$$Q_{\text{abs}} = \frac{C_{\text{abs}}}{\pi a_{\text{eff}}^2}. \quad (11)$$

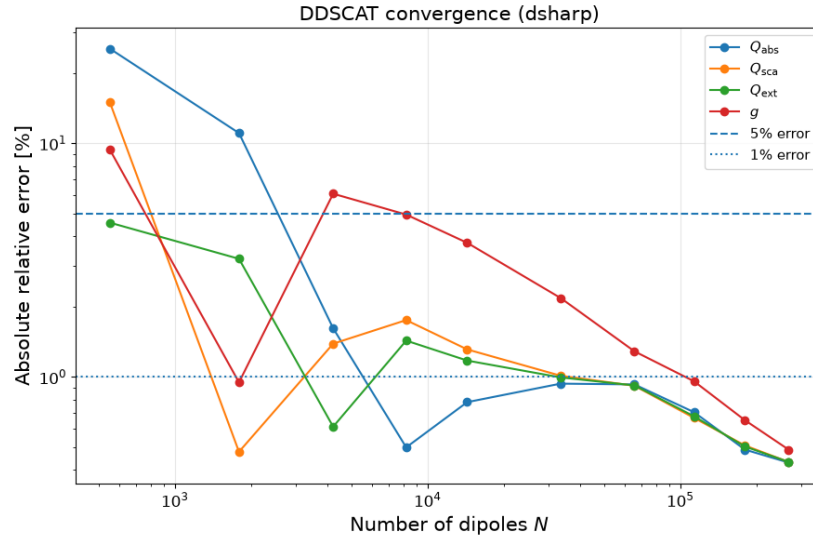


Figure 3: Absolute relative error of the DDSCAT results compared with the Mie reference solution for the DSHARP dust mixture.

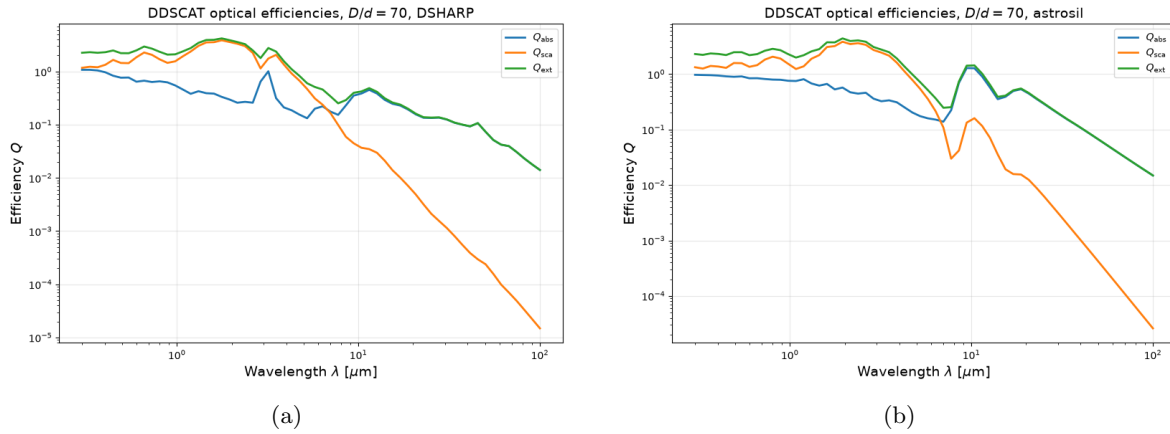
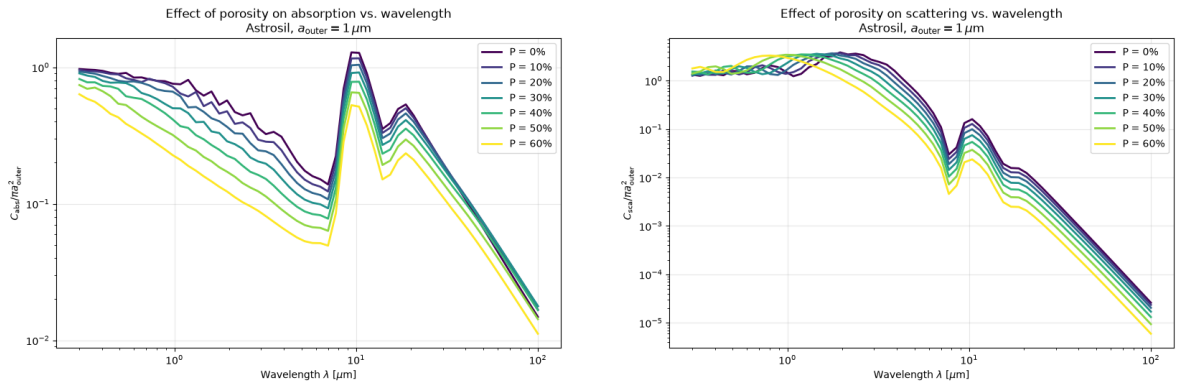


Figure 4: Optical efficiencies of a compact spherical grain with radius $a = 1 \mu\text{m}$ as functions of wavelength, calculated with DDSCAT using a resolution of $D/d = 70$, corresponding to $N = 179\,944$ dipoles. Panel (a) shows the results for the DSHARP dust mixture, and panel (b) shows the results for astronomical silicate. In both cases, the wavelength range is $0.3 \mu\text{m}$ to $100 \mu\text{m}$.

For a direct comparison between grains with the same outer size, it is therefore more useful to scale the absorption cross section with the same outer geometrical cross section,

$$\frac{C_{\text{abs}}}{\pi a_{\text{outer}}^2} = Q_{\text{abs}} \left(\frac{a_{\text{eff}}}{a_{\text{outer}}} \right)^2. \quad (12)$$

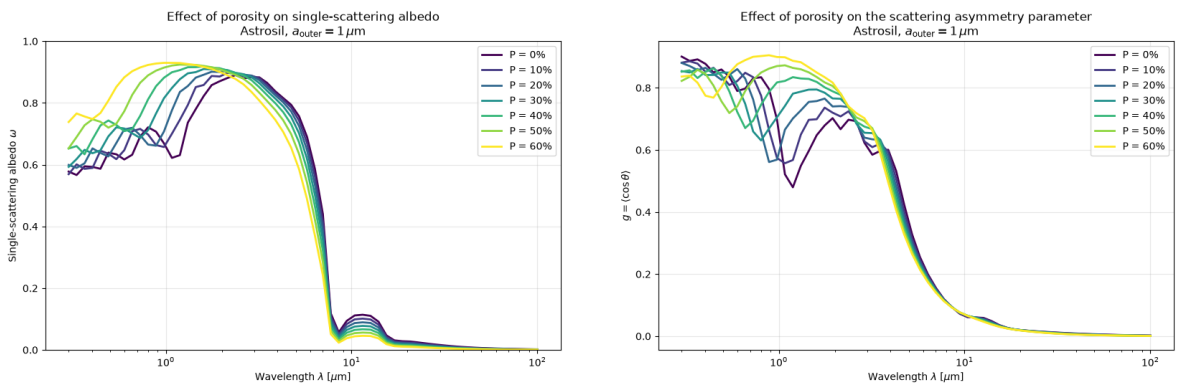
Figure 5 shows the absorption and scattering properties of porous Astrosil grains. At short wavelengths, both absorption and scattering generally decrease with increasing porosity. The compact grain shows stronger small-scale wiggles, while the curves become smoother for higher porosity. The strong feature around $10 \mu\text{m}$ is the silicate feature. Its strength decreases as the porosity increases. A second broader feature is visible around $18\text{--}20 \mu\text{m}$. Figure 6 shows the single-scattering albedo and the asymmetry parameter g . The overall wavelength dependence remains similar for all porosities, which means that porosity mainly changes the strength of the interaction and less the general shape of the curves.



(a) Absorption cross section normalized by πa_{outer}^2 .

(b) Scattering efficiency as a function of wavelength.

Figure 5: Optical efficiencies of porous Astrosil grains with $a_{\text{outer}} = 1 \mu\text{m}$ for porosities between 0% and 60%. (a) Absorption decreases with increasing porosity, especially at short wavelengths. (b) Scattering also becomes weaker for more porous grains.



(a) Single-scattering albedo $\omega = Q_{\text{sca}}/Q_{\text{ext}}$.

(b) Asymmetry parameter $g = \langle \cos \theta \rangle$.

Figure 6: Further optical properties of porous Astrosil grains with $a_{\text{outer}} = 1 \mu\text{m}$. (a) The albedo changes with wavelength and porosity. (b) The asymmetry parameter keeps a similar overall shape, indicating that porosity mainly changes the amplitude of the optical response.

Finally, Fig. 7 shows the absorption cross section relative to the compact grain. The relative difference of Q_{abs} increases with porosity. The absorption efficiency drops for wavelengths between 0.3 and

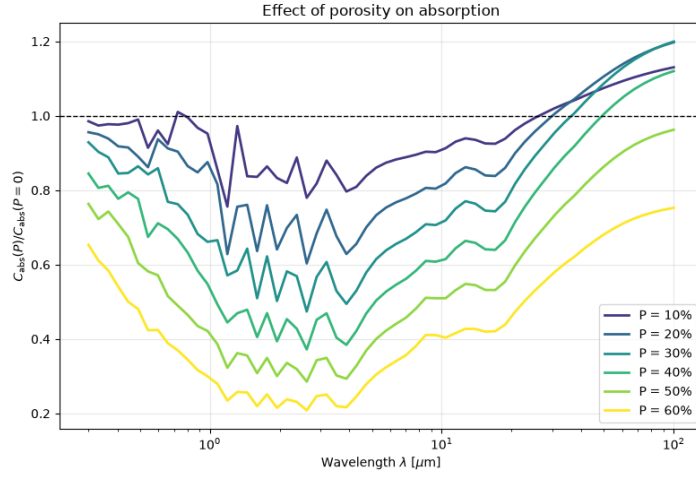


Figure 7: Ratio of the absorption cross section of porous Astrosil grains to that of the compact grain, $C_{\text{abs}}(P)/C_{\text{abs}}(P = 0)$, as a function of wavelength. For most wavelengths the ratio is below one, showing that increasing porosity reduces the absorption of grains with the same outer radius.

The same analysis was also done for a grain with the DSHARP dust composition. The outer envelope was again fixed at $a_{\text{outer}} = 1 \mu\text{m}$, and the same porosities from 0% to 60% were considered. The absorption and scattering cross sections were normalized by the outer geometrical cross section in the same way as for Astrosil.

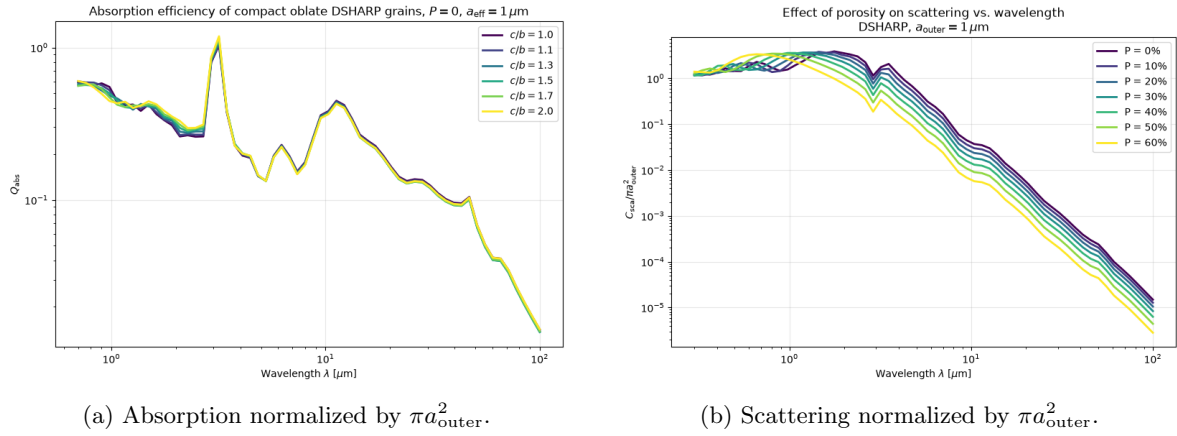


Figure 8: Effect of porosity on the absorption and scattering of DSHARP grains as a function of wavelength. The outer envelope is fixed at $a_{\text{outer}} = 1 \mu\text{m}$.

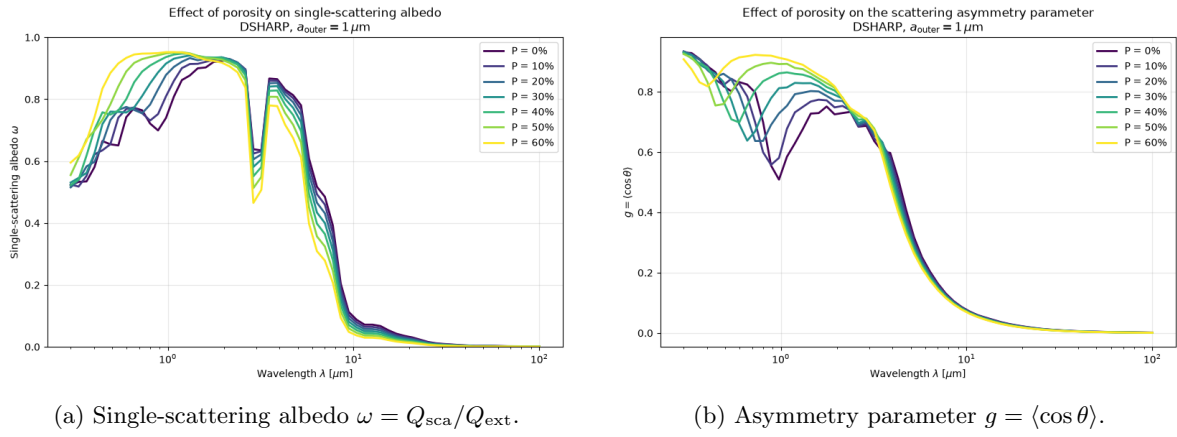


Figure 9: Effect of porosity on the single-scattering albedo and asymmetry parameter of DSHARP grains.

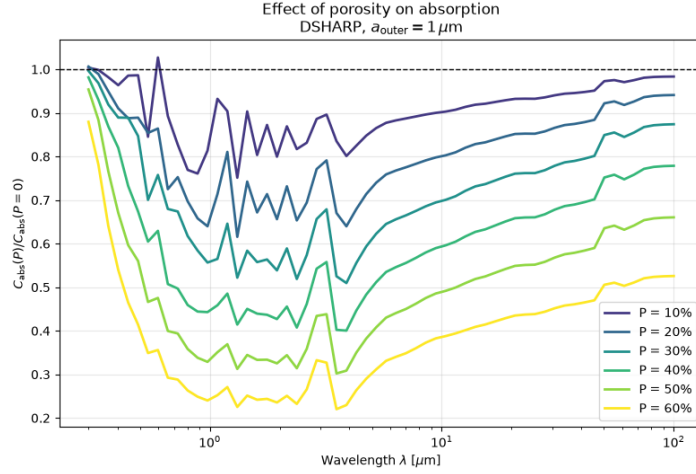


Figure 10: Absorption cross section of porous DSHARP grains relative to the compact grain, $C_{\text{abs}}(P)/C_{\text{abs}}(P=0)$, as a function of wavelength.

2.2.4 Oblate shapes

In addition to porosity, the influence of the grain shape was investigated. Compact oblate grains with porosity $P = 0$ were generated with different axis ratios

$$\frac{c}{b} = 1.0, 1.1, 1.3, 1.5, 1.7, 2.0,$$

where b is the short semi-axis and c is the long semi-axis. An axis ratio of $c/b = 1$ therefore corresponds to a sphere. The effective radius was kept constant at

$$a_{\text{eff}} = (bc^2)^{1/3} = 1 \mu\text{m},$$

so that all grains have approximately the same material volume while only their shape is changed. Since a_{eff} is the same for all grains, the DDSCAT absorption efficiencies Q_{abs} can be compared directly.

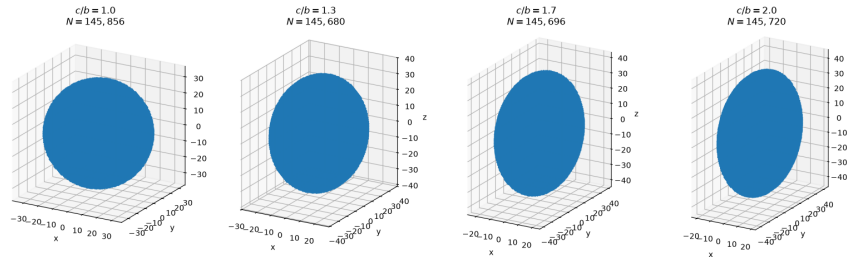


Figure 11: Examples of the compact oblate targets used in the DDSCAT simulations. Increasing c/b corresponds to an increasingly flattened grain, while the effective radius remains constant at $a_{\text{eff}} = 1 \mu\text{m}$.

For Astrosil, changing the axis ratio mainly affects the absorption at short wavelengths and around the spectral features. The overall shape of the absorption spectrum remains similar for all axis ratios. At wavelengths of several tens of micrometres, the differences are smaller, although the most oblate grain ($c/b = 2.0$) shows a somewhat increased absorption compared with the spherical grain.

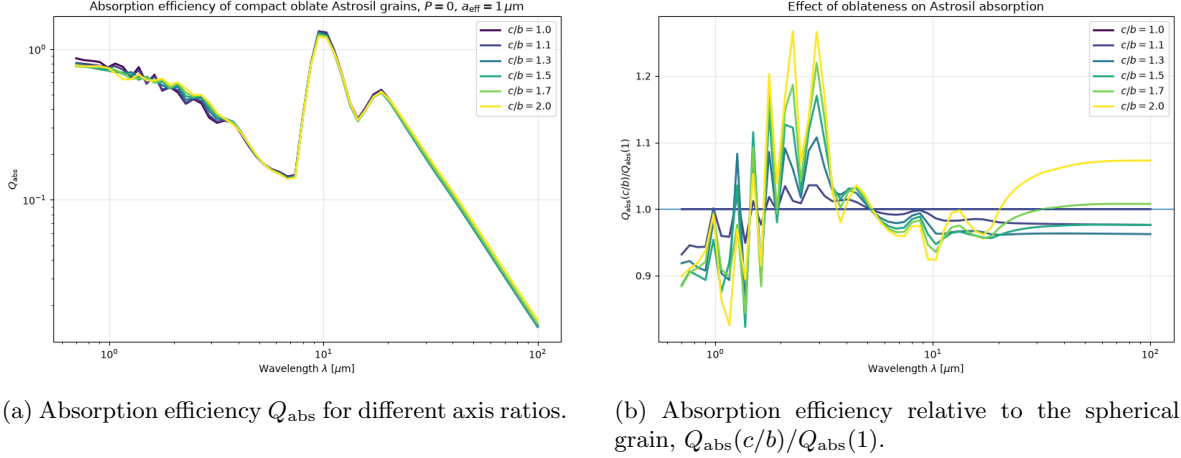


Figure 12: Influence of oblateness on the absorption of compact Astrosil grains with $P = 0$ and $a_{\text{eff}} = 1 \mu\text{m}$.

A similar behaviour is found for the DSHARP composition. The strongest dependence on the axis ratio occurs at shorter wavelengths, where the relative absorption can differ by more than 20% for the most oblate grains. At longer wavelengths the differences become smaller, with the absorption generally remaining within a few percent of the spherical case.

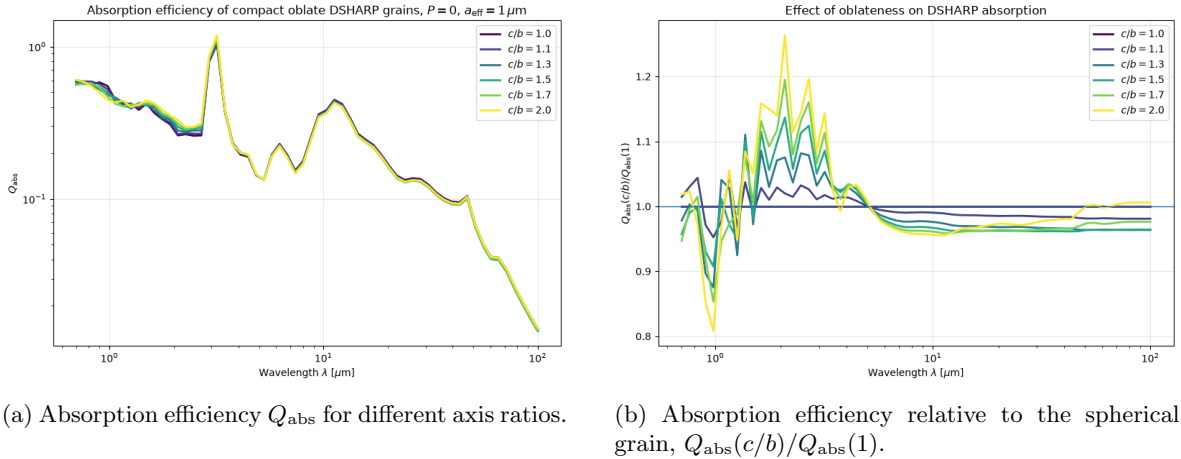


Figure 13: Influence of oblateness on the absorption of compact DSHARP grains with $P = 0$ and $a_{\text{eff}} = 1 \mu\text{m}$.

2.3 DSHARP Analysis

2.3.1 A naive approach

The DSHARP dust model consists of astronomical silicate, refractory organics, troilite (FeS), and water ice. The adopted composition from Birnstiel et al. (2018) is shown in Table 1.

The wavelength-dependent optical constants, $m(\lambda) = n(\lambda) + ik(\lambda)$, were obtained from the material data implemented in `optool`. The optical constants for astronomical silicate are based on Draine [Draine, 2003]. For water-ice, Warren & Brandt [Warren and Brandt, 2008], and Henning & Stognienko [Henning and Stognienko, 1996] for FeS and refractory organics.

Table 1: Composition of the DSHARP dust mixture from Birnstiel et al. [et al., 2018]

Component	Mass fraction	Volume fraction	Density [g cm^{-3}]	Source of n, k data
Water ice	0.2000	0.3642	0.92	Warren & Brandt (2008)
Astronomical silicate	0.3291	0.1670	3.30	Draine (2003)
Troilite (FeS)	0.0743	0.0258	4.83	Henning & Stognienko (1996)
Refractory organics	0.3966	0.4430	1.50	Henning & Stognienko (1996)

To investigate the contribution of the individual materials, the same compact, homogeneous sphere with $P = 0$ and $a_{\text{eff}} = 1 \mu\text{m}$ was used for every component. The grain size and geometry were kept constant and only the material was changed. The absorption efficiency Q_{abs} was calculated between 0.7 and $100 \mu\text{m}$.

DDSCAT converged for astronomical silicate, refractory organics, water ice, and the complete DSHARP mixture. For pure FeS, however, the iterative DDSCAT solver did not converge because of its large refractive index. Since the test grain is a compact homogeneous sphere, the FeS absorption efficiency was therefore calculated using Mie theory.

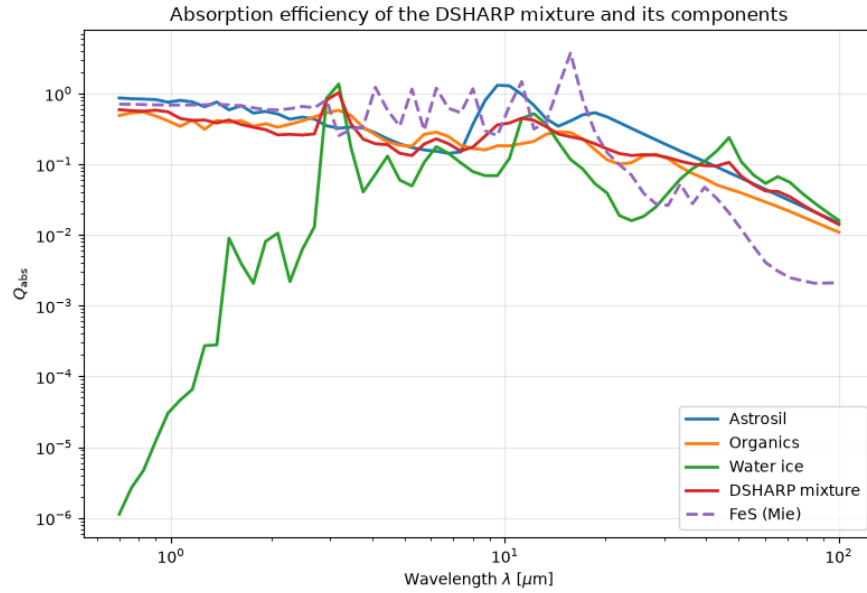


Figure 14: Absorption efficiencies of the individual DSHARP components and the complete mixture. FeS was calculated using Mie theory.

The complete DSHARP mixture is not obtained by directly averaging the absorption efficiencies of the individual materials. As a simple test, a volume-weighted sum was calculated as

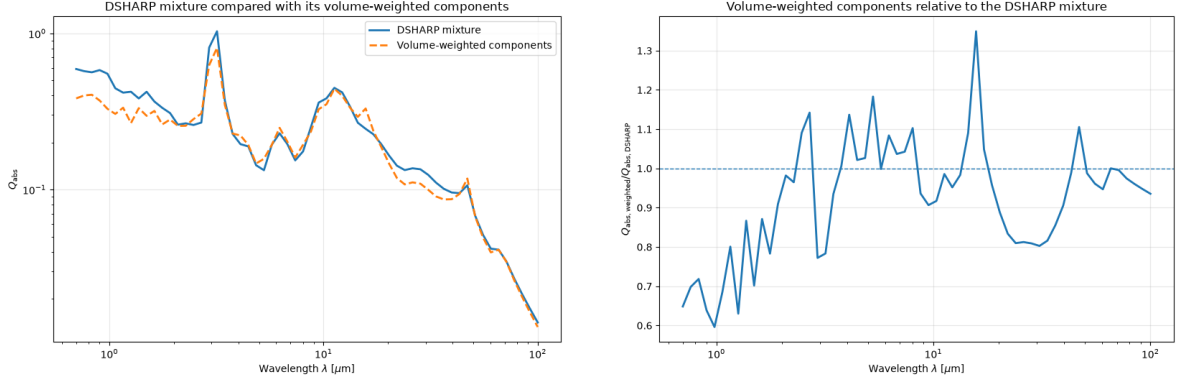
$$Q_{\text{abs,weighted}} = \sum_i f_i Q_{\text{abs},i}, \quad (13)$$

where f_i is the volume fraction of component i .

The current `optool` implementation uses the original DSHARP mass fractions, but a density of 1.4 g cm^{-3} for refractory organics instead of the 1.5 g cm^{-3} used by Birnstiel et al. (2018). For consistency with the `optool` mixture, the resulting volume fractions used here are

$$f_{\text{sil}} = 0.16195, \quad f_{\text{org}} = 0.46004, \quad f_{\text{FeS}} = 0.02498, \quad f_{\text{ice}} = 0.35303.$$

The volume-weighted approach does not reproduce the DSHARP result. The mean absolute difference in Q_{abs} over the investigated wavelength range is 12.69%, with the difference depending strongly on wavelength.



(a) DSHARP compared with the volume-weighted component sum.

(b) Relative difference between the weighted sum and DSHARP.

Figure 15: Comparison of the DSHARP mixture with a simple volume-weighted combination of its components.

2.3.2 Bruggeman rule

In `optool`, the DSHARP mixture is instead calculated using the Bruggeman effective-medium rule. The complex refractive index of each component,

$$m_i = n_i + ik_i,$$

is first converted to its complex dielectric function,

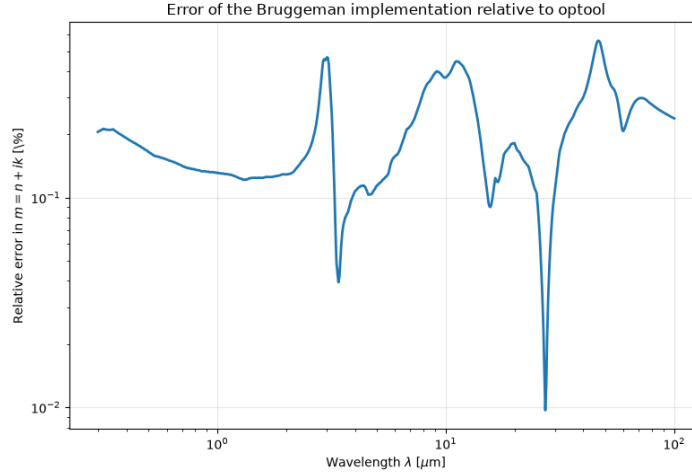
$$\epsilon_i = m_i^2.$$

The effective dielectric function ϵ_{eff} of the mixture is then obtained by solving

$$\sum_i f_i \frac{\epsilon_i - \epsilon_{\text{eff}}}{\epsilon_i + 2\epsilon_{\text{eff}}} = 0. \quad (14)$$

This relation accounts for the fact that the components are embedded in and electromagnetically interact with an effective surrounding medium. The local electric field and polarization therefore depend on the other materials in the mixture. Consequently, mixing the dielectric properties is not equivalent to linearly combining the final absorption efficiencies.

To test this, the Bruggeman equation was implemented independently and the resulting effective refractive index was compared with the DSHARP optical constants produced by `optool`. The mean relative difference in the complex refractive index was only 0.203%. This is much smaller than the 12.69% mean absolute difference obtained with the simple volume-weighted Q_{abs} approach and confirms that the Bruggeman mixing rule reproduces the DSHARP effective optical properties well.

(a) Relative error of the implemented Bruggeman mixture compared with `optool`.

2.4 Combined Oblateness and Porosity

Finally, the effects of porosity and grain shape were investigated together. Oblate grains with axis ratios $c/b = 1.1, 1.3, 1.5, 1.7$, and 2.0 were calculated for porosities between $P = 0$ and $P = 0.6$.

To compare the influence of porosity for the different shapes, the absorption of the highly porous grains with $P = 0.6$ was compared with the corresponding compact grains with $P = 0$. Since removing material reduces the DDSCAT effective radius, the absorption was normalized to the same reference area of the compact parent grain.

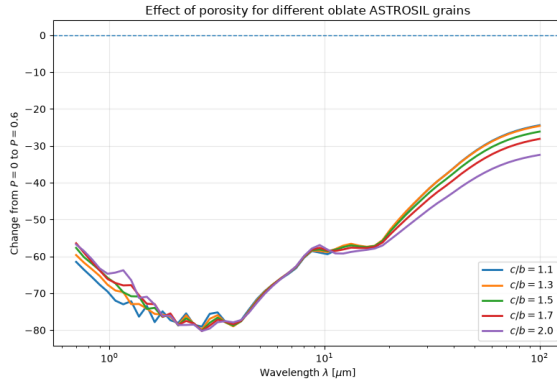
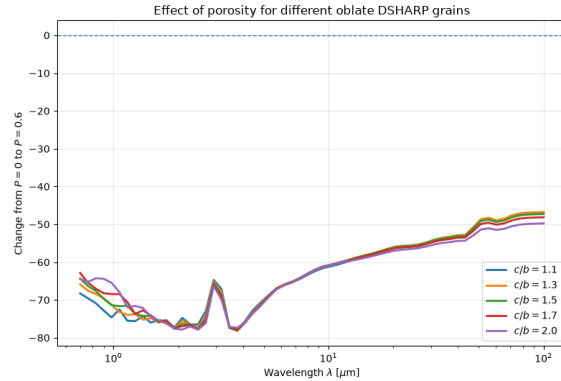
(a) Relative change from $P = 0$ to $P = 0.6$ for different Astrosil oblate shapes.(b) Relative change from $P = 0$ to $P = 0.6$ for different DSHARP oblate shapes.

Figure 17: Effect of increasing the porosity from $P = 0$ to $P = 0.6$ for different oblate grain shapes.

For DSHARP grains, the mean absolute change between $P = 0$ and $P = 0.6$ is about 63% for all investigated axis ratios. For Astrosil grains, the mean absolute change is about 57–59%. The differences between the individual oblate shapes are therefore small compared with the overall effect of porosity.

At $\lambda \approx 10 \mu\text{m}$, the absorption decreases by about 61% for DSHARP and by about 58–59% for Astrosil when the porosity is increased from $P = 0$ to $P = 0.6$. Only a weak dependence on the axis ratio is visible.

This suggests that, within the investigated parameter range, porosity has a much stronger influence

on the absorption than the degree of oblateness. The effect of increasing the porosity remains nearly the same when the axis ratio is changed from $c/b = 1.1$ to $c/b = 2.0$.

2.5 Different grain sizes

The influence of grain size was investigated for compact, homogeneous spheres with radii

$$a = 0.005, 0.01, 0.05, 0.1, 0.5, 1 \mu\text{m}.$$

The calculations were performed for both Astrosil and the DSHARP dust mixture over the wavelength range from 0.3 to 100 μm . For all grain sizes, the same relative geometrical resolution of approximately $D/d = 70$ was used. Thus, the number of dipoles remained approximately constant, while the physical dipole spacing scaled with the grain size. The grains were compact spheres with $P = 0$, so that $a_{\text{eff}} = a$.

The absorption efficiency is shown in Fig. 18. For both materials, Q_{abs} generally increases with increasing grain radius. For the smaller grains, the curves have a similar wavelength dependence and mainly differ in their overall absorption strength. As the grain size increases, deviations from this simple scaling become more visible and the material-dependent spectral features become more pronounced.

For Astrosil, the strong silicate feature around $\lambda \approx 10 \mu\text{m}$ is clearly visible for all investigated sizes and becomes particularly strong for the larger grains. The DSHARP mixture shows a broader and more complex wavelength dependence because it contains several different materials.

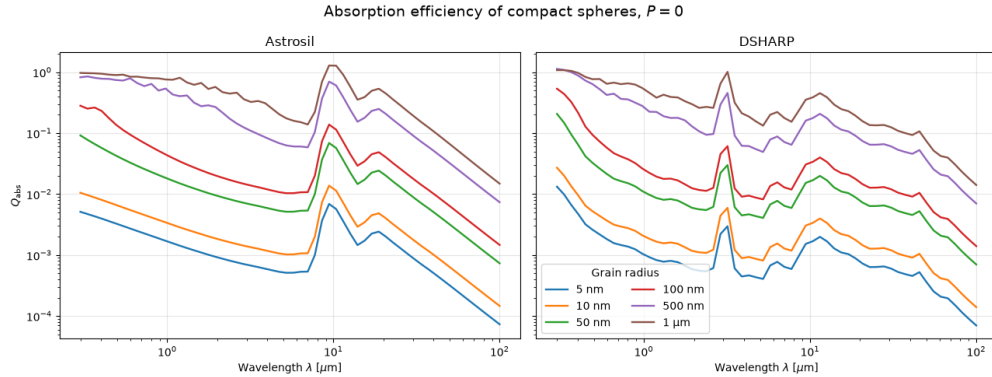


Figure 18: Absorption efficiency of compact Astrosil and DSHARP spheres for grain radii between 5 nm and 1 μm .

The size dependence can also be seen by considering Q_{abs} at fixed wavelengths. In general, the absorption efficiency increases with grain radius for all investigated wavelengths. At wavelengths much larger than the grain size, the grains are close to the Rayleigh regime and the increase with radius is approximately regular. At larger size parameters, the behaviour becomes more strongly dependent on wavelength and material composition.

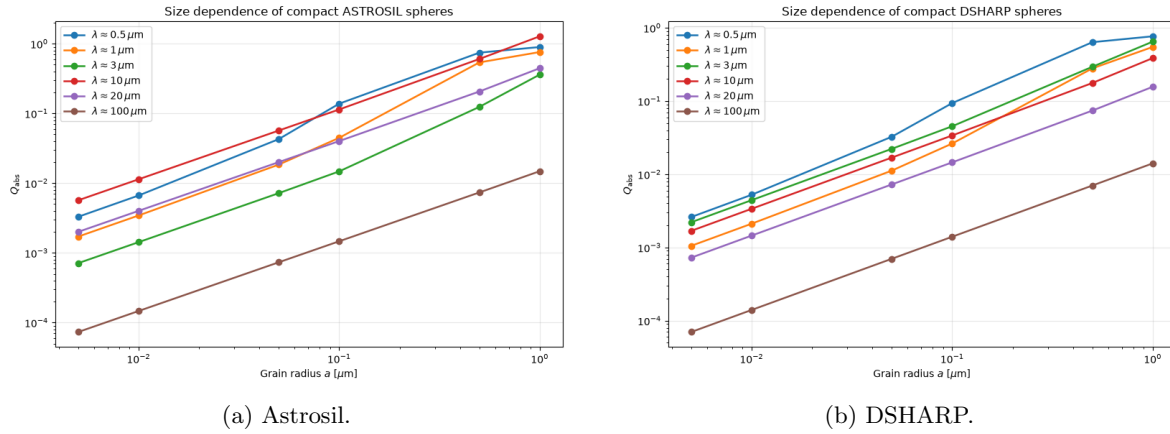


Figure 19: Absorption efficiency as a function of grain radius at selected wavelengths.

2.6 Two materials in one grain

As an additional test, compact spherical grains consisting of astronomical silicate and water ice were considered. The outer radius was kept at $a = 1 \mu\text{m}$ and the same resolution as for the compact sphere was used, corresponding to $N = 179944$ occupied dipoles. Instead of removing dipoles, as in the porosity study, all grid positions remained occupied and were assigned either astronomical silicate or water ice. The water-ice volume fraction was varied from 0 to 60% in steps of 10%, with the ice dipoles distributed randomly throughout the grain.

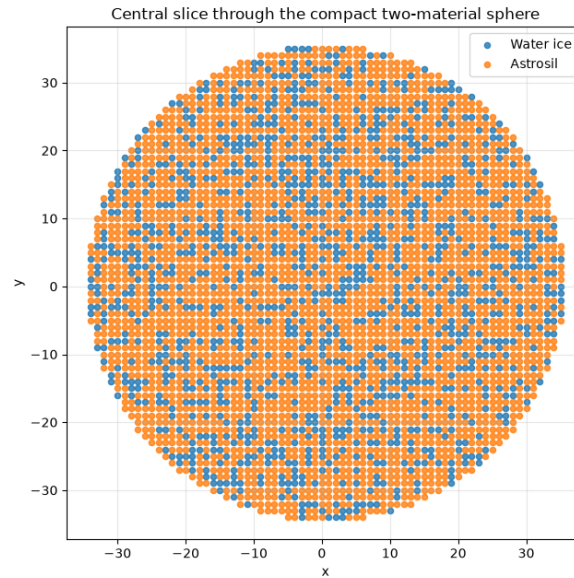


Figure 20: Central slice through a compact two-material grain. Each point represents one occupied DDSCAT dipole and the colors indicate whether it contains astronomical silicate or water ice.

The explicit DDSCAT calculations were compared with the Bruggeman effective-medium approximation. For each material, the complex dielectric function was calculated from the refractive index as $\epsilon = (n + ik)^2$. The effective dielectric function was then obtained from

$$(1 - f_{\text{ice}}) \frac{\epsilon_{\text{sil}} - \epsilon_{\text{eff}}}{\epsilon_{\text{sil}} + 2\epsilon_{\text{eff}}} + f_{\text{ice}} \frac{\epsilon_{\text{ice}} - \epsilon_{\text{eff}}}{\epsilon_{\text{ice}} + 2\epsilon_{\text{eff}}} = 0, \quad (15)$$

where f_{ice} is the water-ice volume fraction. The corresponding effective refractive index was calculated as $m_{\text{eff}} = \sqrt{\epsilon_{\text{eff}}}$. This refractive index was used with `optool` to calculate Q_{abs} for a homogeneous sphere with the same radius.

Figure 21 shows the effect of increasing the water-ice fraction in the explicit DDSCAT grains. Adding water ice generally reduces the absorption over large parts of the wavelength range, although the effect is strongly wavelength dependent and some spectral features become enhanced.

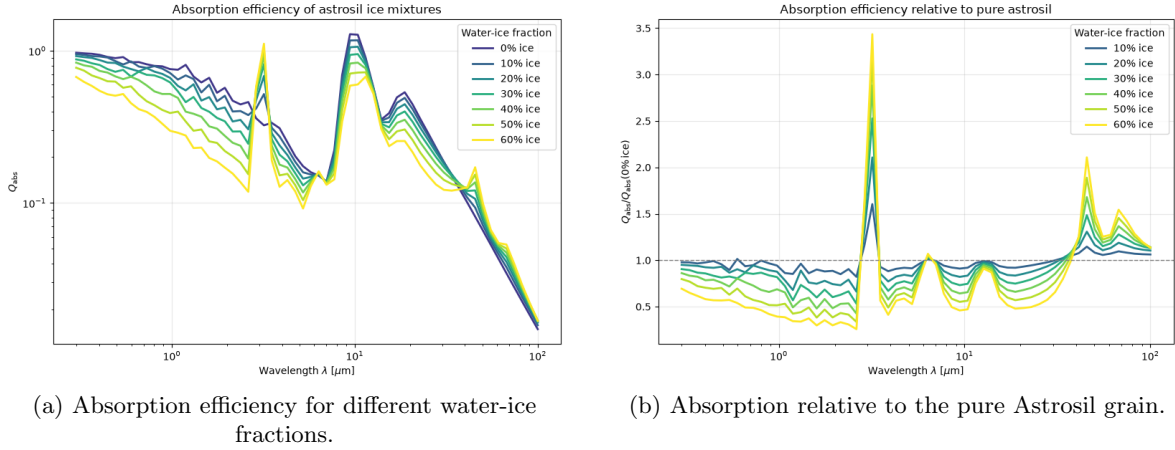


Figure 21: Effect of the water-ice fraction on the absorption of the explicit two-material DDSCAT grains. (a) Absolute absorption efficiency Q_{abs} . (b) Absorption efficiency normalized to the pure Astrosil grain.

To compare the explicit mixtures directly with the corresponding Bruggeman models, the ratio

$$R(\lambda) = \frac{Q_{\text{abs,explicit}}(\lambda)}{Q_{\text{abs,Bruggeman}}(\lambda)} \quad (16)$$

was calculated separately for every water-ice fraction. A value of $R = 1$ therefore represents perfect agreement between the two calculations, while values below or above one indicate lower or higher absorption in the explicit DDSCAT grain, respectively.

The agreement varies with wavelength and becomes weaker as the water-ice fraction increases. For the pure Astrosil grain, the remaining difference is small and mainly reflects the numerical difference between DDSCAT and the homogeneous `optool` calculation. At higher ice fractions, larger deviations occur at individual wavelengths.

To describe the overall agreement with a single number, the relative difference was defined as

$$\Delta Q_{\text{abs}}(\lambda) = 100 \frac{Q_{\text{abs,explicit}}(\lambda) - Q_{\text{abs,Bruggeman}}(\lambda)}{Q_{\text{abs,Bruggeman}}(\lambda)}, \quad (17)$$

and the mean absolute relative difference over all wavelengths was calculated as

$$|\overline{\Delta Q_{\text{abs}}}| = \frac{1}{N_{\lambda}} \sum_{\lambda} |\Delta Q_{\text{abs}}(\lambda)|. \quad (18)$$

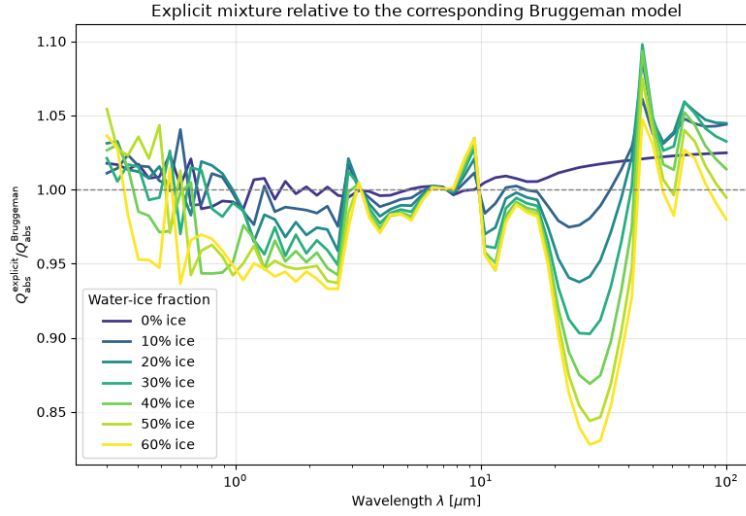


Figure 22: Ratio of the explicit DDSCAT absorption efficiency to the corresponding Bruggeman effective-medium result. Each curve compares models with the same water-ice volume fraction.

The mean absolute difference increases almost monotonically from 1.06% for pure Astrosil to 1.62%, 2.42%, 3.03%, 3.77%, 4.29%, and 4.57% for water-ice fractions from 10 to 60%. At 60% ice, the median difference is 3.42%, while the maximum wavelength-dependent difference reaches 17.21%. Thus, the homogeneous Bruggeman approximation becomes gradually less representative of the explicitly mixed grain as the fraction of water ice increases. Nevertheless, the mean difference remains below 5% for all mixtures considered.

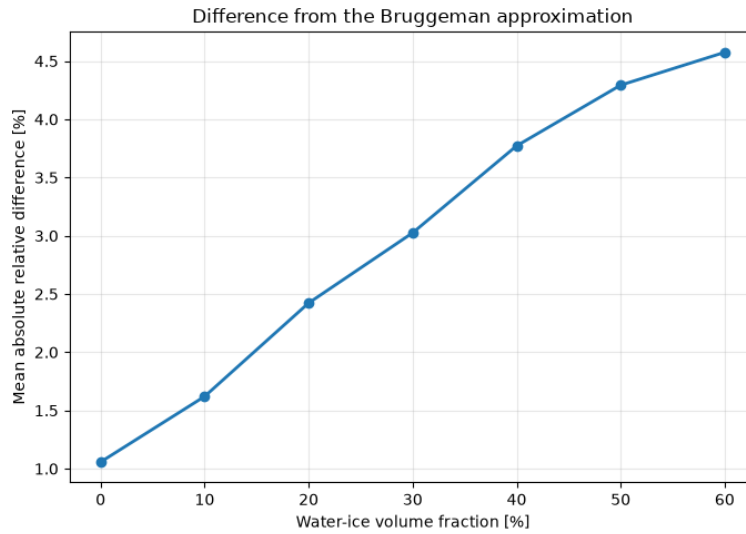


Figure 23: Mean absolute relative difference in Q_{abs} between the explicit DDSCAT grains and the corresponding Bruggeman models as a function of water-ice volume fraction.

3 Conclusion

During this project, I became familiar with the discrete dipole approximation and learned how to set up, run, and evaluate DDSCAT calculations for different dust-grain geometries and compositions. An important part of the work was the development of a reliable numerical workflow: first validating the method against Mie theory, then testing the required dipole resolution, and finally applying the method to more complex targets. I also gained experience in Python programming, automated generation of DDSCAT input files, analysis of large sets of output data, and the implementation of the Bruggeman effective-medium rule.

The simulations showed that the optical properties of dust grains depend strongly on wavelength, composition, size, porosity, and, to a smaller extent in the parameter range studied here, on grain shape. Increasing the porosity generally reduced the absorption and scattering for grains with the same outer size, while changing the axis ratio mainly modified individual spectral features. Grain size had a strong influence on the overall absorption efficiency. For the explicit Astrosil–water-ice mixtures, the agreement with the homogeneous Bruggeman approximation became gradually weaker as the ice fraction increased. However, even for 60% water ice, the mean absolute difference in Q_{abs} remained below 5%, although deviations at individual wavelengths reached about 17%.

These results are qualitatively consistent with previous studies showing that porosity and internal grain structure can significantly modify dust absorption and scattering and that these effects are wavelength dependent Voshchinnikov et al. [2006], Shen et al. [2008]. Shen et al. also found that effective-medium models can reproduce DDA calculations reasonably well in many cases, while deviations increase when the internal grain structure becomes more important. The exact quantitative results are not directly comparable because the grain geometries, compositions, and normalization used in the studies are different. :contentReference[oaicite:0]index=0

A possible continuation of this work would be to look at larger and more dust grain populations instead of individual particles. This could include distributions of grain sizes, porosities, shapes and compositions. Scattering and polarization could also be studied using the Mueller matrices calculated by DDSCAT. Combining the optical properties of such grain populations would make it possible to calculate synthetic extinction, scattering, polarization, or thermal-emission spectra. These models could then be compared with astronomical observations and may help to constrain properties such as the size, composition, and internal structure of observed dust grains.

References

- B. T. Draine. Scattering by interstellar dust grains. i. optical and ultraviolet. *The Astrophysical Journal*, 598:1017–1025, 2003. doi: 10.1086/379118.
- Birnstiel et al. The disk substructures at high angular resolution project (dsharp). v. interpreting alma maps of protoplanetary disks in terms of a dust model. *The Astrophysical Journal Letters*, 869:L45, 2018. doi: 10.3847/2041-8213/aaf743.
- Th. Henning and R. Stognienko. Dust opacities for protoplanetary accretion disks: Influence of dust aggregates. *Astronomy & Astrophysics*, 311:291–303, 1996.
- Yue Shen, B. T. Draine, and Eric T. Johnson. Modeling porous dust grains with ballistic aggregates. i. geometry and optical properties. *The Astrophysical Journal*, 689:260–275, 2008. doi: 10.1086/592765.
- N. V. Voshchinnikov, V. B. Il’in, Th. Henning, and D. N. Dubkova. Dust extinction and absorption: the challenge of porous grains. *Astronomy & Astrophysics*, 445:167–177, 2006. doi: 10.1051/0004-6361:20053371.
- Stephen G. Warren and Richard E. Brandt. Optical constants of ice from the ultraviolet to the microwave: A revised compilation. *Journal of Geophysical Research: Atmospheres*, 113:D14220, 2008. doi: 10.1029/2007JD009744.

4 Appendix

4.1 Python code used in the analysis

This appendix contains the main Python routines used for the numerical analysis in this report. Repeated plotting commands and cluster submission code are omitted where the same procedure was used for several figures.

4.1.1 Reading DDSCAT output and comparison with Mie theory

The main wavelength-dependent quantities were read from `qtable`.

```

1 import numpy as np
2
3
4 def read_qtable(filename):
5     rows = []
6
7     with open(filename) as file:
8         for line in file:
9             values = line.split()
10
11             if len(values) < 8:
12                 continue
13
14             try:
15                 rows.append(
16                     [float(x) for x in values[:8]]
17                 )
18             except ValueError:
19                 continue
20
21     return np.array(rows)
22
23
24 data = read_qtable(qtable)
25
26 a_eff = data[:, 0]
27 wavelength = data[:, 1]
28 qext = data[:, 2]
29 qabs = data[:, 3]
30 qsca = data[:, 4]
31 g = data[:, 5]
```

Listing 1: Reading a DDSCAT `qtable`.

For compact homogeneous spheres, the optool mass opacities were converted to efficiency factors using

$$Q = \frac{4}{3} \rho a \kappa.$$

The absolute relative error between DDSCAT and the Mie solution was then calculated for each quantity.

```
1 import numpy as np
2
3
4 def opacity_to_q(
5     kappa,
6     density,
7     radius_um
8 ):
9     radius_cm = (
10         radius_um
11         * 1e-4
12     )
13
14     return (
15         4.0 / 3.0
16         * density
17         * radius_cm
18         * kappa
19     )
20
21
22 def absolute_relative_error(
23     ddscat,
24     mie
25 ):
26     return (
27         100.0
28         * np.abs(
29             ddscat - mie
30         )
31         / np.abs(mie)
32     )
33
34
35 qabs_mie = opacity_to_q(
36     kappa_abs,
37     density,
38     1.0
39 )
40
41 qsca_mie = opacity_to_q(
42     kappa_sca,
43     density,
44     1.0
45 )
46
47 qext_mie = (
48     qabs_mie
49     + qsca_mie
50 )
51
52
53 error_qabs = absolute_relative_error(
54     qabs_ddscat,
55     qabs_mie
56 )
57
58 error_qsca = absolute_relative_error(
59     qsca_ddscat,
60     qsca_mie
61 )
62
63 error_qext = absolute_relative_error(
64     qext_ddscat,
65     qext_mie
66 )
```

```

67
68 error_g = absolute_relative_error(
69     g_ddscat,
70     g_mie
71 )

```

Listing 2: Conversion of optool opacities and calculation of the DDSCAT–Mie error.

The same error calculation was applied to the convergence calculations with different dipole numbers. The physical radius and wavelength were kept fixed while only the target resolution was changed.

4.1.2 Generation of porous spherical targets

The compact parent sphere had an outer radius of 35 dipole spacings, corresponding to approximately $D/d = 70$. Porosity was generated by randomly removing dipoles from this compact parent target.

```

1  from pathlib import Path
2  import numpy as np
3
4
5  OUTPUT = Path(
6      POROSITY_TARGETS
7  )
8
9  RADIUS = 35
10
11  POROSITIES = [
12      0.0,
13      0.1,
14      0.2,
15      0.3,
16      0.4,
17      0.5,
18      0.6,
19  ]
20
21  SEED = 20260902
22
23  A_OUTER_UM = 1.0
24
25
26  OUTPUT.mkdir(
27      parents=True,
28      exist_ok=True
29  )
30
31
32  compact_coordinates = []
33
34
35  for ix in range(-34, 36):
36      for iy in range(-34, 36):
37          for iz in range(-34, 36):
38
39              r2 = (
40                  (ix - 0.5)**2
41                  + (iy - 0.5)**2
42                  + (iz - 0.5)**2
43              )
44
45              if r2 <= RADIUS**2:
46                  compact_coordinates.append(
47                      (ix, iy, iz)
48                  )
49

```

```
50
51 compact_coordinates = np.array(
52     compact_coordinates,
53     dtype=int
54 )
55
56 n_compact = len(
57     compact_coordinates
58 )
59
60
61 rng = np.random.default_rng(
62     SEED
63 )
64
65 order = rng.permutation(
66     n_compact
67 )
68
69
70 def write_shape(
71     filename,
72     coordinates
73 ):
74
75     with open(
76         filename,
77         'w'
78     ) as file:
79
80         file.write(
81             'Porous spherical target\n'
82         )
83
84         file.write(
85             f'{len(coordinates)} = NAT\n'
86         )
87
88         file.write(
89             '1.0 0.0 0.0 = A1\n'
90         )
91
92         file.write(
93             '0.0 1.0 0.0 = A2\n'
94         )
95
96         file.write(
97             '1.0 1.0 1.0 = lattice spacings\n'
98         )
99
100        file.write(
101            '0.5 0.5 0.5 = X0\n'
102        )
103
104        file.write(
105            'JA IX IY IZ\n'
106            'ICOMPX ICOMPY ICOMPZ\n'
107        )
108
109
110        for ja, (
111            ix,
112            iy,
113            iz
114        ) in enumerate(
115            coordinates,
116            start=1
```

```

117         ):
118
119             file.write(
120                 f{ja}
121                 f{ix} {iy} {iz}
122                 1 1 1\n
123             )
124
125
126 for porosity in POROSITIES:
127
128     n_keep = round(
129         (1.0 - porosity)
130         * n_compact
131     )
132
133     coordinates = (
134         compact_coordinates[
135             order[:n_keep]
136         ]
137     )
138
139
140     label = (
141         fP{int(round(
142             100 * porosity
143         )):02d}
144     )
145
146
147     directory = (
148         OUTPUT
149         / label
150     )
151
152     directory.mkdir(
153         exist_ok=True
154     )
155
156
157     write_shape(
158         directory
159         / shape.dat,
160         coordinates
161     )
162
163
164     a_eff_um = (
165         A_OUTER_UM
166         * (
167             n_keep
168             / n_compact
169         )**(1.0 / 3.0)
170     )
171
172
173     print(
174         label,
175         N =,
176         n_keep,
177         a_eff =,
178         a_eff_um
179     )

```

Listing 3: Generation of porous spherical shape.dat targets.

4.1.3 Normalization of porous-grain cross sections

DDSCAT normalizes its efficiency factors using the effective radius a_{eff} . For the porosity comparison, the outer radius was fixed, so the cross sections were normalized by the same outer geometrical area.

```

1 a_outer = 1.0
2
3
4 a_eff = data[:, 0]
5
6 qext = data[:, 2]
7 qabs = data[:, 3]
8 qsca = data[:, 4]
9 g = data[:, 5]
10
11
12 area_factor = (
13     a_eff
14     / a_outer
15 )**2
16
17
18 cabs_over_outer_area = (
19     qabs
20     * area_factor
21 )
22
23 csca_over_outer_area = (
24     qsca
25     * area_factor
26 )
27
28
29 single_scattering_albedo = (
30     qsca
31     / qext
32 )

```

Listing 4: Normalization of porous-grain optical quantities.

The absorption relative to the compact grain was calculated as

```

1 relative_absorption = (
2     cabs_over_outer_area_por
3     / cabs_over_outer_area_compact
4 )

```

4.1.4 Generation of compact oblate targets

For an axis ratio $q = c/b$, where c is the long semi-axis and b the short semi-axis, the semi-axes were chosen according to

$$b = \frac{R_{\text{eq}}}{q^{2/3}}, \quad c = R_{\text{eq}} q^{1/3},$$

so that the material volume remains approximately constant.

```

1 from pathlib import Path
2 import numpy as np
3
4
5 OUTPUT = Path(
6     OBLATE_TARGETS

```

```

7 )
8
9 AXIS_RATIOS = [
10     1.0,
11     1.1,
12     1.3,
13     1.5,
14     1.7,
15     2.0,
16 ]
17
18
19 R_EQ = 32.65
20
21
22 OUTPUT.mkdir(
23     parents=True,
24     exist_ok=True
25 )
26
27
28 def write_shape(
29     filename,
30     coordinates
31 ):
32
33     with open(
34         filename,
35         'w'
36     ) as file:
37
38         file.write(
39             'Oblate target\n'
40         )
41
42         file.write(
43             f'{len(coordinates)} = NAT\n'
44         )
45
46         file.write(
47             '1.0 0.0 0.0 = A1\n'
48         )
49
50         file.write(
51             '0.0 1.0 0.0 = A2\n'
52         )
53
54         file.write(
55             '1.0 1.0 1.0
56             = lattice spacings\n'
57         )
58
59         file.write(
60             '0.5 0.5 0.5 = X0\n'
61         )
62
63         file.write(
64             'JA IX IY IZ
65             ICOMPX ICOMPY ICOMPZ\n'
66         )
67
68
69         for ja, (
70             ix,
71             iy,
72             iz
73         ) in enumerate(

```

```

74         coordinates,
75         start=1
76     ):
77
78         file.write(
79             f{ja}
80             f{ix} {iy} {iz}
81             1 1 1\n
82         )
83
84
85 for ratio in AXIS_RATIOS:
86
87     b = (
88         R_EQ
89         / ratio**(
90             2.0 / 3.0
91         )
92     )
93
94     c = (
95         R_EQ
96         * ratio**(
97             1.0 / 3.0
98         )
99     )
100
101
102     limit = (
103         int(
104             np.ceil(c)
105         )
106         + 2
107     )
108
109
110     coordinates = []
111
112
113     for ix in range(
114         -limit,
115         limit + 1
116     ):
117         for iy in range(
118             -limit,
119             limit + 1
120         ):
121             for iz in range(
122                 -limit,
123                 limit + 1
124             ):
125
126                 value = (
127                     (
128                         (ix - 0.5)
129                         / c
130                     )**2
131                     + (
132                         (iy - 0.5)
133                         / c
134                     )**2
135                     + (
136                         (iz - 0.5)
137                         / b
138                     )**2
139                 )
140

```

```

141         if value <= 1.0:
142             coordinates.append(
143                 (
144                     ix,
145                     iy,
146                     iz
147                 )
148             )
149
150
151 coordinates = np.array(
152     coordinates,
153     dtype=int
154 )
155
156
157 label = (
158     fAR{int(round(
159         10 * ratio
160     )):02d}
161 )
162
163
164 directory = (
165     OUTPUT
166     / label
167 )
168
169 directory.mkdir(
170     exist_ok=True
171 )
172
173
174 write_shape(
175     directory
176     / shape.dat,
177     coordinates
178 )
179
180
181 print(
182     label,
183     c/b =,
184     ratio,
185     N =,
186     len(coordinates)
187 )

```

Listing 5: Generation of compact oblate targets.

4.1.5 Combined oblateness and porosity

For the combined analysis, the porous oblate targets were generated using the same random-removal procedure. The relative change between $P = 0$ and $P = 0.6$ was calculated as follows.

```

1 relative_change = (
2     100.0
3     * (
4         cabs_p60
5         - cabs_p00
6     )
7     / cabs_p00
8 )
9
10

```

```

11 mean_absolute_change = np.mean(
12     np.abs(
13         relative_change
14     )
15 )
16
17
18 index_10um = np.argmin(
19     np.abs(
20         wavelength
21         - 10.0
22     )
23 )
24
25
26 change_at_10um = (
27     relative_change[
28         index_10um
29     ]
30 )

```

4.1.6 DSHARP optical constants

The optical constants of the individual materials were read from the material files.

```

1 from pathlib import Path
2 import numpy as np
3
4
5 base = Path(
6     /star/data/skugler/
7     DDSCAT/
8     DSHARP_COMPONENTS/
9     RUNS
10 )
11
12
13 def read_diel(
14     filename
15 ):
16     return np.loadtxt(
17         filename,
18         skiprows=3
19     )
20
21
22 sil = read_diel(
23     base
24     / ASTROSIL
25     / diel.dat
26 )
27
28 org = read_diel(
29     base
30     / ORGANICS
31     / diel.dat
32 )
33
34 fes = read_diel(
35     base
36     / FES
37     / diel.dat
38 )
39
40 ice = read_diel(
41     base

```

```

42     / ICE
43     / diel.dat
44 )
45
46 dsharp = read_diel(
47     base
48     / DSHARP
49     / diel.dat
50 )
51
52
53 wavelength = sil[:, 0]

```

4.1.7 Conversion from mass fractions to volume fractions

```

1 mass_fractions = np.array([
2     0.3291,
3     0.3966,
4     0.0743,
5     0.2000,
6 ])
7
8
9 densities = np.array([
10    3.30,
11    1.40,
12    4.83,
13    0.92,
14 ])
15
16
17 relative_volumes = (
18     mass_fractions
19     / densities
20 )
21
22
23 volume_fractions = (
24     relative_volumes
25     / relative_volumes.sum()
26 )
27
28
29 print(
30     volume_fractions
31 )

```

This gives approximately

$$f_{\text{sil}} = 0.16195, \quad f_{\text{org}} = 0.46004, \quad f_{\text{FeS}} = 0.02498, \quad f_{\text{ice}} = 0.35303.$$

4.1.8 Bruggeman effective-medium calculation

```

1 from scipy.optimize import root
2 import numpy as np
3
4
5 materials = [
6     sil,
7     org,
8     fes,
9     ice,
10 ]

```

```
11
12
13 epsilon = []
14
15
16 for material in materials:
17
18     n = material[:, 1]
19     k = material[:, 2]
20
21     m = (
22         n
23         + 1j * k
24     )
25
26     epsilon.append(
27         m**2
28     )
29
30
31 epsilon = np.array(
32     epsilon
33 )
34
35
36 def bruggeman_residual(
37     x,
38     eps_components,
39     fractions
40 ):
41
42     eps_eff = (
43         x[0]
44         + 1j * x[1]
45     )
46
47
48     value = 0j
49
50
51     for fraction, eps_i in zip(
52         fractions,
53         eps_components
54     ):
55
56         value += (
57             fraction
58             * (
59                 eps_i
60                 - eps_eff
61             )
62             / (
63                 eps_i
64                 + 2 * eps_eff
65             )
66         )
67
68
69     return [
70         value.real,
71         value.imag
72     ]
73
74
75 epsilon_mix = np.zeros(
76     len(wavelength),
77     dtype=complex
```

```

78 )
79
80
81 guess = np.sum(
82     volume_fractions[:, None]
83     * epsilon,
84     axis=0
85 )[0]
86
87
88 for j in range(
89     len(wavelength)
90 ):
91     solution = root(
92         bruggeman_residual,
93         [
94             guess.real,
95             guess.imag
96         ],
97         args=(
98             epsilon[:, j],
99             volume_fractions
100         )
101     )
102
103
104
105 if not solution.success:
106     raise RuntimeError(
107         solution.message
108     )
109
110
111 epsilon_mix[j] = (
112     solution.x[0]
113     + 1j * solution.x[1]
114 )
115
116
117 guess = (
118     epsilon_mix[j]
119 )
120
121
122 m_mix = np.sqrt(
123     epsilon_mix
124 )
125
126
127 n_mix = (
128     m_mix.real
129 )
130
131 k_mix = np.abs(
132     m_mix.imag
133 )

```

4.1.9 Comparison with the optool DSHARP mixture

```

1 m_bruggeman = (
2     n_mix
3     + 1j * k_mix
4 )
5
6
7 m_optool = (

```

```
8     dsharp[:, 1]
9     + 1j * dsharp[:, 2]
10 )
11
12
13 relative_error_bruggeman = (
14     100.0
15     * np.abs(
16         m_bruggeman
17         - m_optool
18     )
19     / np.abs(
20         m_optool
21     )
22 )
23
24
25 mean_error = np.mean(
26     relative_error_bruggeman
27 )
28
29
30 maximum_error = np.max(
31     relative_error_bruggeman
32 )
33
34
35 print(
36     Mean error:,
37     mean_error
38 )
39
40 print(
41     Maximum error:,
42     maximum_error
43 )
```

4.1.10 Naive volume-weighted absorption

```
1 qabs_weighted = (
2     volume_fractions[0]
3     * qabs_astrosil
4     + volume_fractions[1]
5     * qabs_organics
6     + volume_fractions[2]
7     * qabs_fes
8     + volume_fractions[3]
9     * qabs_ice
10 )
11
12
13 relative_difference = (
14     100.0
15     * (
16         qabs_weighted
17         - qabs_dsharp
18     )
19     / qabs_dsharp
20 )
21
22
23 mean_absolute_difference = np.mean(
24     np.abs(
25         relative_difference
26     )
27 )
```

```

28
29
30 maximum_absolute_difference = np.max(
31     np.abs(
32         relative_difference
33     )
34 )

```

4.1.11 FeS Mie calculation

For pure FeS, the DDSCAT solver did not converge. The Mie result was therefore converted from opacity to absorption efficiency.

```

1 a_cm = 1e-4
2
3 rho_fes = 4.83
4
5
6 qabs_fes = (
7     4.0 / 3.0
8     * rho_fes
9     * a_cm
10    * kappa_abs_fes
11 )

```

4.1.12 Different grain sizes

The grain-size study did not require an additional numerical routine. The same `qtable` reader was used for every radius. The physical radius was changed in the DDSCAT input while the relative resolution was kept at approximately $D/d = 70$.

4.1.13 Explicit Astrosil–water-ice grains

The two-material targets used the same compact $D/d = 70$ spherical lattice. No dipoles were removed. Instead, every occupied dipole was assigned either material 1 (Astrosil) or material 2 (water ice).

```

1 from pathlib import Path
2 import numpy as np
3
4
5 OUTPUT = Path(
6     ASTROSIL_ICE/
7     EXPLICIT
8 )
9
10
11 ICE_FRACTIONS = [
12     0.0,
13     0.1,
14     0.2,
15     0.3,
16     0.4,
17     0.5,
18     0.6,
19 ]
20
21
22 RADIUS = 35
23
24 SEED = 20260902
25
26

```

```

27 OUTPUT.mkdir(
28     parents=True,
29     exist_ok=True
30 )
31
32
33 coordinates = []
34
35
36 for ix in range(-34, 36):
37     for iy in range(-34, 36):
38         for iz in range(-34, 36):
39
40             r2 = (
41                 (ix - 0.5)**2
42                 + (iy - 0.5)**2
43                 + (iz - 0.5)**2
44             )
45
46             if r2 <= RADIUS**2:
47                 coordinates.append(
48                     (
49                         ix,
50                         iy,
51                         iz
52                     )
53                 )
54
55
56 coordinates = np.array(
57     coordinates,
58     dtype=int
59 )
60
61
62 n_dipoles = len(
63     coordinates
64 )
65
66
67 rng = np.random.default_rng(
68     SEED
69 )
70
71
72 order = rng.permutation(
73     n_dipoles
74 )
75
76
77 def write_two_material_shape(
78     filename,
79     coordinates,
80     material
81 ):
82
83     with open(
84         filename,
85         'w'
86     ) as file:
87
88         file.write(
89             'Astrosil-water-ice sphere\n'
90         )
91
92         file.write(
93             f'{len(coordinates)} = NAT\n'

```

```

94         )
95
96         file.write(
97             1.0 0.0 0.0 = A1\n
98         )
99
100        file.write(
101            0.0 1.0 0.0 = A2\n
102        )
103
104        file.write(
105            1.0 1.0 1.0
106            = lattice spacings\n
107        )
108
109        file.write(
110            0.5 0.5 0.5 = X0\n
111        )
112
113        file.write(
114            JA IX IY IZ
115            ICOMPX ICOMPY ICOMPZ\n
116        )
117
118
119        for ja, (
120            (
121                ix,
122                iy,
123                iz
124            ),
125            mat
126        ) in enumerate(
127            zip(
128                coordinates,
129                material
130            ),
131            start=1
132        ):
133
134            file.write(
135                f{ja}
136                f{ix} {iy} {iz}
137                f{mat} {mat} {mat}\n
138            )
139
140
141        for fraction in ICE_FRACTIONS:
142
143            n_ice = round(
144                fraction
145                * n_dipoles
146            )
147
148
149            material = np.ones(
150                n_dipoles,
151                dtype=int
152            )
153
154
155            material[
156                order[:n_ice]
157            ] = 2
158
159
160            label = (

```

```

161         fF{int(round(
162             100 * fraction
163         )):02d}
164     )
165
166
167     directory = (
168         OUTPUT
169         / label
170     )
171
172     directory.mkdir(
173         exist_ok=True
174     )
175
176
177     write_two_material_shape(
178         directory
179         / shape.dat,
180         coordinates,
181         material
182     )

```

The corresponding DDSCAT parameter file contained two dielectric materials,

```

1 2 = NCOMP
2 'astrosil.dat'
3 'ice.dat'

```

so that 1 1 1 and 2 2 2 in `shape.dat` refer to Astrosil and water ice, respectively.

4.1.14 Reading DDSCAT material files

The material files used for the two-component calculation did not all have the same column order. Therefore, the column definitions in the DDSCAT header were read explicitly.

```

1 import numpy as np
2
3
4 def read_ddscat_material(
5     filename
6 ):
7
8     with open(
9         filename
10     ) as file:
11
12         lines = (
13             file.readlines()
14         )
15
16
17         columns = (
18             lines[1]
19             .split(=)[0]
20             .split()
21         )
22
23
24         wave_col = (
25             int(columns[0])
26             - 1
27         )
28
29         n_col = (

```

```
30         int(columns[1])
31         - 1
32     )
33
34     k_col = (
35         int(columns[2])
36         - 1
37     )
38
39
40     rows = []
41
42
43     for line in lines[3:]:
44
45         values = (
46             line.split()
47         )
48
49         if not values:
50             continue
51
52
53         try:
54
55             rows.append(
56                 [
57                     float(x)
58                     for x in values
59                 ]
60             )
61
62         except ValueError:
63
64             continue
65
66
67     rows = np.array(
68         rows
69     )
70
71
72     return (
73         rows[:, wave_col],
74         rows[:, n_col],
75         rows[:, k_col]
76     )
```

4.1.15 Two-component Bruggeman calculation

```
1  from scipy.optimize import root
2  import numpy as np
3
4
5  wave_sil, n_sil, k_sil = (
6      read_ddscat_material(
7          astrosil
8      )
9  )
10
11
12  wave_ice, n_ice, k_ice = (
13      read_ddscat_material(
14          ice.dat
15      )
16  )
```

```
17
18
19 n_sil_i = np.interp(
20     np.log(wavelength),
21     np.log(wave_sil),
22     n_sil
23 )
24
25 k_sil_i = np.interp(
26     np.log(wavelength),
27     np.log(wave_sil),
28     k_sil
29 )
30
31 n_ice_i = np.interp(
32     np.log(wavelength),
33     np.log(wave_ice),
34     n_ice
35 )
36
37 k_ice_i = np.interp(
38     np.log(wavelength),
39     np.log(wave_ice),
40     k_ice
41 )
42
43
44 eps_sil = (
45     n_sil_i
46     + 1j * k_sil_i
47 )**2
48
49
50 eps_ice = (
51     n_ice_i
52     + 1j * k_ice_i
53 )**2
54
55
56 def two_component_bruggeman(
57     eps1,
58     eps2,
59     fraction_2
60 ):
61
62     eps_eff = np.zeros(
63         len(eps1),
64         dtype=complex
65     )
66
67
68     guess = (
69         (1.0 - fraction_2)
70         * eps1[0]
71         + fraction_2
72         * eps2[0]
73     )
74
75
76     for j in range(
77         len(eps1)
78     ):
79
80         def residual(x):
81
82             value = (
83                 x[0]
```

```

84         + 1j * x[1]
85     )
86
87
88     equation = (
89         (1.0 - fraction_2)
90         * (
91             eps1[j]
92             - value
93         )
94         / (
95             eps1[j]
96             + 2.0 * value
97         )
98         + fraction_2
99         * (
100             eps2[j]
101             - value
102         )
103         / (
104             eps2[j]
105             + 2.0 * value
106         )
107     )
108
109
110     return [
111         equation.real,
112         equation.imag
113     ]
114
115
116     solution = root(
117         residual,
118         [
119             guess.real,
120             guess.imag
121         ]
122     )
123
124
125     if not solution.success:
126         raise RuntimeError(
127             solution.message
128         )
129
130
131     eps_eff[j] = (
132         solution.x[0]
133         + 1j * solution.x[1]
134     )
135
136
137     guess = (
138         eps_eff[j]
139     )
140
141
142     m_eff = np.sqrt(
143         eps_eff
144     )
145
146
147     return (
148         m_eff.real,
149         np.abs(
150             m_eff.imag

```

```

151     )
152 )

```

4.1.16 Writing the Bruggeman mixture for optool

```

1 def write_optool_lnk(
2     filename,
3     wavelength,
4     n,
5     k
6 ):
7
8     with open(
9         filename,
10        w
11    ) as file:
12
13        file.write(
14            # Bruggeman
15            Astrosil-water-ice mixture\n
16        )
17
18
19        file.write(
20            f{len(wavelength)} 1.0\n
21        )
22
23
24        for (
25            wave,
26            n_value,
27            k_value
28        ) in zip(
29            wavelength,
30            n,
31            k
32        ):
33
34            file.write(
35                f{wave:.8e}
36                f{n_value:.8e}
37                f{k_value:.8e}\n
38            )

```

4.1.17 Explicit DDSCAT versus Bruggeman comparison

```

1 ratio = (
2     qabs_explicit
3     / qabs_bruggeman
4 )
5
6
7 relative_difference = (
8     100.0
9     * (
10        qabs_explicit
11        - qabs_bruggeman
12    )
13    / qabs_bruggeman
14 )
15
16
17 mean_absolute_difference = np.mean(
18     np.abs(

```

```
19         relative_difference
20     )
21 )
22
23
24 median_absolute_difference = np.median(
25     np.abs(
26         relative_difference
27     )
28 )
29
30
31 maximum_absolute_difference = np.max(
32     np.abs(
33         relative_difference
34     )
35 )
36
37
38 print(
39     Mean absolute difference:,
40     mean_absolute_difference
41 )
42
43 print(
44     Median absolute difference:,
45     median_absolute_difference
46 )
47
48 print(
49     Maximum absolute difference:,
50     maximum_absolute_difference
51 )
```

4.1.18 Visualisation of custom targets

A central slice through `shape.dat` was used to check the target geometry and the distribution of different materials.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 data = np.loadtxt(
6     shape.dat,
7     skiprows=7
8 )
9
10
11 x = data[:, 1]
12 y = data[:, 2]
13 z = data[:, 3]
14
15 material = (
16     data[:, 4]
17     .astype(int)
18 )
19
20
21 slice_mask = (
22     z == 0
23 )
24
25
26 plt.figure(
```

```
27     figsize=(7, 7)
28 )
29
30
31 for material_number in np.unique(
32     material
33 ):
34
35     selection = (
36         slice_mask
37         & (
38             material
39             == material_number
40         )
41     )
42
43
44     plt.scatter(
45         x[selection],
46         y[selection],
47         s=8,
48         label=(
49             f'Material
50             f'{material_number}'
51         )
52     )
53
54
55 plt.xlabel(x)
56 plt.ylabel(y)
57
58 plt.axis(equal)
59 plt.legend()
60 plt.tight_layout()
61 plt.show()
```